

Reviewer Pack — Minimal Order of Formal Grammar Entailment and SAT Clause En...

Entry 4cd75d03e19a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 18:52:40 UTC

Minimal Order of Formal Grammar Entailment and SAT Clause Entropy Inequality

Entry ID: 4cd75d03e19a **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 18:52:40 UTC

1. Conjecture

Field A (mathematical branch): Formal Logic (Entailment Theory) **Field B** (complexity object): Boolean Satisfiability (SAT Clause Entropy)

Statement:

For every Boolean formula φ with n clauses, the minimal order of formal grammar entailments in φ is linearly correlated with its SAT clause entropy, such that $\text{MinEnt}(\varphi) = \Theta(\text{Entropy}(\varphi))$.

Rationale (proposer's reasoning):

Formal logic, particularly entailment theory, provides a framework for reasoning about logical relationships. This conjecture suggests that the complexity of these relationships can be captured by the information content (entropy) in the clauses of the SAT formula, which could expose a deeper connection between logical structure and computational complexity.

Taxonomy category: Formal Logic × Boolean Satisfiability (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2e01c61338d7f1cc

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If, for a given Boolean formula φ with n clauses, the Pearson correlation coefficient between $\text{MinEnt}(\varphi)$ and $\text{Entropy}(\varphi)$ is ≥ 0.8 AND no seed produces an absolute difference of $\text{MinEnt}(\varphi)$ from $\text{Entropy}(\varphi)$ greater than 3 over all measured formulas.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - formal logic AND entailment theory AND 'minimal order' AND Boolean satisfiability AND 'SAT clause entropy' - Boolean satisfiability AND SAT clause entropy AND formal grammar AND 'entailment inequality' - formal logic AND 'minimal order of grammar' AND correlation WITH Boolean satisfiability AND 'entropy inequality'

Top relevant hits considered: - [http://arxiv.org/abs/1905.01545v1] A Logic Framework for P2P Deductive Databases - [http://arxiv.org/abs/1309.6883v2] Predicate Logic as a Modeling Language: Modeling and Solving some Machine Learning and Data Mining Problems with IDP3 - [http://arxiv.org/abs/1912.00010v1] A Restricted Second-Order Logic for Non-deterministic Poly-Logarithmic Time - [http://arxiv.org/abs/1004.0653v2] Exact Ramsey Theory: Green-Tao numbers and SAT - [http://arxiv.org/abs/2605.09347v1] Dsat: A Native SAT Solver for Discrete Logic - [http://arxiv.org/abs/1505.03340v2] HordeSat: A Massively Parallel Portfolio SAT Solver - [http://arxiv.org/abs/1411.6452v3] Modal Extensions of Łukasiewicz Logic for Modeling Coalitional Power - [http://arxiv.org/abs/1912.03841v4] A Logic that Captures β P on Ordered Structures

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(n):
        clauses = []
        for _ in range(n):
            literals = [f"x{i}" if random.choice([True, False]) else f"~x{i}" for i in range(1, n+1)]
            clause = " & ".join(literals)
            clauses.append(clause)
        formula = " | ".join(clauses)
        return formula

    def compute_min_ents(formula):
        # Placeholder for actual computation
        # For simplicity, we'll assume a linear relationship here
        min_ents = [random.uniform(1, 5) for _ in range(n)]
        return min_ents

    def compute_entropy(clauses):
        entropy = sum(-len(clause.split(' & ')) * math.log2(len(clause.split(' & '))) for clause in clauses)
```

```

    return entropy

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    formula = generate_formula(n)
    min_ents = compute_min_ents(formula)
    entropies = [compute_entropy(clause.split(' | ')) for clause in formula.split(' | ')]

    if len(min_ents) != len(entropies):
        return {
            "metric_name": "Pearson Correlation Coefficient",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "Mismatch in number of min_ents and entropies"
        }

    results.append({
        "min_ents": min_ents,
        "entropies": entropies
    })

return {
    "metric_name": "Pearson Correlation Coefficient",
    "metric_value": None,
    "instances_tested": len(n_values),
    "n_max": max(n_values),
    "conjecture_holds": False,
    "counterexample": "Not enough data to compute correlation"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all("metric_value" not in r or r["metric_value"] is None for r in results):
        support_fraction = sum(1 for result in results if abs(result["metric_value"]) >= 0.8 and a
    else:
        support_fraction = None

    if support_fraction is not None and support_fraction >= 0.8:
        RESULT = "SUPPORTED"
    elif any("counterexample" in r and r["counterexample"] for r in results):
        counterexample = next(r["counterexample"] for r in results if "counterexample" in r)
        RESULT = f"FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seeds[results
    else:

```

```

RESULT = "INCONCLUSIVE insufficient_data"

print(RESULT)

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

'counterexample': 'Not enough data to compute correlation'}
TRIAL: {'metric_name': 'Pearson Correlation Coefficient', 'metric_value': None, 'instances_tested': 0}
TRIAL: {'metric_name': 'Pearson Correlation Coefficient', 'metric_value': None, 'instances_tested': 0}
TRIAL: {'metric_name': 'Pearson Correlation Coefficient', 'metric_value': None, 'instances_tested': 0}
TRIAL: {'metric_name': 'Pearson Correlation Coefficient', 'metric_value': None, 'instances_tested': 0}
TRIAL: {'metric_name': 'Pearson Correlation Coefficient', 'metric_value': None, 'instances_tested': 0}
TRIAL: {'metric_name': 'Pearson Correlation Coefficient', 'metric_value': None, 'instances_tested': 0}
TRIAL: {'metric_name': 'Pearson Correlation Coefficient', 'metric_value': None, 'instances_tested': 0}
TRIAL: {'metric_name': 'Pearson Correlation Coefficient', 'metric_value': None, 'instances_tested': 0}
TRIAL: {'metric_name': 'Pearson Correlation Coefficient', 'metric_value': None, 'instances_tested': 0}
TRIAL: {'metric_name': 'Pearson Correlation Coefficient', 'metric_value': None, 'instances_tested': 0}

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it was unable to compute the Pearson correlation coefficient and verify the conjecture. | next: Investigate the cause of the crash in the test code and attempt to run the test again with a different approach or improved error handling.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17380
2	preregistration	ollama_remote	glm4:latest	0	0	9552
3	novelty	ollama_remote	glm4:latest	0	0	8700
4	novelty	ollama_remote	glm4:latest	0	0	9529
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13370
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10698
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11001
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9505
9	judge	ollama_remote	glm4:latest	0	0	12312

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 102048 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/4cd75d03e19a.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/4cd75d03e19a.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4cd75d03e19a.tar.gz (if generated)